



TRABALHO

SISTEMA DE AGENDAMENTO DE EVENTOS COM RESTRIÇÕES

Enzo Rocha, Giovana Leite e Murilo Amaral



Conteúdo

1

Apresentação das linguagens

2

Diferenciais entre as linguagens

3

Apresentação do problema

4

Apresentação da solução

5

Comparativo dos resultados

6

Considerações finais

As linguagens escolhidas

Prolog



Coomon LISP



Ruby





O que é o Prolog?

1

Prolog utiliza uma base de conhecimento (BC) composta por átomos, fatos e regras.

2

Os átomos representam elementos simbólicos, e os fatos e regras definem relações entre eles.

3

Ao receber uma consulta, a linguagem busca na BC e pode retornar respostas verdadeiras, falsas ou valores que satisfaçam a consulta.



O que é o Common Lisp?

1

Lisp utiliza átomos e listas para estruturar dados e código

2

É uma das principais representantes do paradigma funcional de programação, no qual tudo é feito utilizando funções.

3

Feita para computação simbólica, sendo muito usada para o desenvolvimento de Inteligências artificiais



O que é o Ruby Programming Language?

1

Código limpo e intuitivo que parece inglês comum.

2

Forte suporte a iteradores dinâmicos nativos, que reduzem a necessidade de laços de repetição

3

Milhares de bibliotecas prontas (gems) para uso imediato.

Principais diferenças

Há algumas diferenças fundamentais entre as linguagens escolhidas, que são mostradas na tabela abaixo

Linguagem	Paradigma Principal	Como Resolver os Problemas?	Unidade Básica	Controle de Fluxo
Prolog	Lógico/Declarativo	Buscando fatos e deduzindo respostas (Inferência).	O Predicado (Fatos e Regras)	Backtracking (tentativa e erro automático).
Lisp	Funcional	Avaliando funções matemáticas aninhadas.	A Lista	Recursão (funções chamando a si mesmas).
Ruby	Orientação a Objetos/Imperativo	Alterando o estado dos objetos passo a passo.	O Objeto	Laços (while, .each) e Condicionais.

Nosso objetivo

O objetivo é solucionar o problema de agendamento de monitorias em ambientes acadêmicos, onde múltiplas restrições devem ser consideradas simultaneamente, como disponibilidade dos professores, conflitos de horário, limite de atendimentos e compatibilidade entre disciplinas. A solução desenvolvida permite validar essas regras de forma automatizada, garantindo a consistência dos agendamentos e evitando conflitos de agenda para alunos e professores.

**CONSULTA
HORÁRIOS**

**AGENDA
HORÁRIO**

**VERIFICAÇÃO DE
CONFLITOS**

**AGENDA/ ERRO DE
AGENDAMENTO**

Como nossa solução funciona?

1

Consulta: o aluno consulta os horários disponíveis dos professores.

2

Validação: o sistema verifica todas as regras de negócio e informa em caso de rejeição.

3

Agendamento: se todas as validações forem aprovadas, a monitoria é registrada no sistema.

Detalhes da Implementação (nosso readme.md)

Requisitos de implementação

- Informações dos professores e monitoria (nome, disciplina ministrada e horário disponível para monitoria);
- Informações dos alunos (nome, curso e semestre matriculado);
- Agendamentos (Alunos que já marcaram horário com os professores).

Regras de negócio

- Disponibilidade do professor;
- Conflito de agenda do professor;
- Conflito de agenda do aluno;
- Limite semanal (2 agendamento por aluno);
- Compatibilidade da disciplina.

Funcionalidades mínimas

- Consultar horários disponíveis de um professor;
- Validar todas as regras de negócio;
- Registrar um novo agendamento válido;

Implementação em Prolog

- **Representação Declarativa:** Professores, alunos, horários e agendamentos são modelados como fatos da base de conhecimento, permitindo que os dados sejam consultados diretamente por meio de inferências lógicas.
 - **Regras como Predicados:** As restrições de negócio são implementadas através de predicados, tornando a validação das regras próxima de sua descrição natural e facilitando a manutenção do sistema.
 - **Inferência Automática:** O mecanismo de resolução do Prolog realiza buscas e verificações automaticamente, eliminando a necessidade de estruturas explícitas de controle como laços de repetição e condicionais complexas.
 - **Backtracking e Consultas Lógicas:** A linguagem explora alternativas automaticamente durante as consultas, permitindo verificar disponibilidade, conflitos e restrições de forma concisa e eficiente.
- 

1. Base de conhecimento (BC)

Como as bases de conhecimento são representados em Prolog.

```
1 professor(tinos, estruturas_de_dados).
2 professor(baranauskas, paradigmas_de_programacao).
3 professor(joaquim, banco_de_dados).
4
5 aluno(ana, man, 3).
6 aluno(pedro, bcc, 5).
7
8 disponivel(tinos, segunda, '14h').
9 disponivel(tinos, quarta, '14h').
10
11 agendamento(ana, tinos, estruturas_de_dados, segunda, '14h').
```

2. Conflito de agenda

Exemplo de validação de conflito de agenda usando lógica declarativa.

```
1 validar_professor_livre(Professor, Dia, Hora) :-  
2     \+ agendamento(_, Professor, _, Dia, Hora), !.  
3  
4 validar_professor_livre(_, _, _) :-  
5     writeln('Inválido: horário já ocupado.'),  
6     fail.
```


3. Processo de agendamento

Encadeamento das regras de negócio e atualização da BC.

```
1 agendar(Aluno, Professor, Disciplina, Dia, Hora) :-  
2  
3     validar_disciplina(Professor, Disciplina),  
4     validar_disponibilidade(Professor, Dia, Hora),  
5     validar_professor_livre(Professor, Dia, Hora),  
6     validar_aluno_livre(Aluno, Dia, Hora),  
7     validar_limite_semanal(Aluno),  
8  
9     assertz(  
10         agendamento(  
11             Aluno,  
12             Professor,  
13             Disciplina,  
14             Dia,  
15             Hora  
16         )  
17     ),  
18  
19     writeln('Agendamento aprovado.').
```

3. Listar agendamento

Regra utilizada para retornar os agendamentos para o usuario

```
1 listar_agendamentos :-  
2     forall(  
3         agendamento(  
4             Aluno,  
5             Professor,  
6             Disciplina,  
7             Dia,  
8             Hora  
9         ),  
10    format(  
11        '~w | ~w | ~w | ~w | ~w~n',  
12        [Aluno, Professor, Disciplina, Dia, Hora]  
13    )  
14 ).
```

Implementação em Common Lisp

- **Flexibilidade de Tipo:** Alunos, professores e a agenda são declarados apenas como variáveis globais, que depois se tornam listas de listas.
- **Paradigma Funcional:** Toda a lógica e manipulação dos dados é feita através de funções.
- **Código como Dados:** Código e dados compartilham a mesma aparência física (listas). Isso permite que o sistema lide com agendas e comparações (usando comandos como equal e first) com lógica simbólica.
- **Iteração Funcional:** É usado o comando dolist ao invés de loops baseados em index para checar a disponibilidade na agenda, permitindo uma abordagem mais limpa e declarativa para percorrer uma estrutura de dados.



Lisp

1. Variáveis Globais

Variáveis globais para representar alunos, professores e a agenda



```
1 (defparameter *aluno* nil)
2 (defparameter *professor* nil)
3 (defparameter *agenda* nil)
```

Lisp

2. Manipulação de Listas

Uso de funções para adicionar listas de dados nas variáveis globais

```
1 ; função para adicionar aluno
2 (defun add_aluno (nome curso periodo)
3   (setf *aluno* (cons (list nome curso periodo @) *aluno*)))
4
5 ; função para adicionar professor
6 (defun add_prof (nome disciplina)
7   (setf *professor* (cons (list nome disciplina) *professor*)))
8
9 ; função para registrar um horário de atendimento
10 (defun cadastrar-monitoria (prof disciplina dia hora)
11   (setf *agenda* (cons (list prof disciplina dia hora "disponivel") *agenda*)))
```

Lisp

3. Agendamento com listas

Uso de listas para verificar os requisitos de agendamento

```
1 (dolist (x *agenda*)
2   (if (equal x (list prof disciplina dia hora "disponivel"))
3       (progn
4         (setf (fifth x) aluno)
5         (setf ok 1))))
```

Implementação em Ruby

- **Orientação a Objetos:** Separação clara de responsabilidades. Os dados ficam isolados na classe Agendamento e a inteligência na classe GerenciadorDCM.
- **Performance com Hashes:** Uso de dicionários para mapear professores e disciplinas. Garante validações de acesso instantâneo na memória, eliminando buscas lentas.
- **Filosofia Fail-Fast:** Validação estruturada com Cláusulas de Guarda. O código testa as falhas primeiro e aborta imediatamente, evitando blocos de if/else aninhados e confusos.
- **Iteradores Nativos:** Substituição de laços manuais (for/while) por métodos de alta ordem (como .find para achar conflitos e .count para checar limites semanais), tornando o código limpo e declarativo.



Ruby

1. Orientação a Objetos

Uso de dicionários para mapear professores e disciplinas.

```
1 class Agendamento
2   # Define leitura pública, mantendo as variáveis privadas
3   attr_reader :aluno, :professor, :disciplina, :dia, :horario
4
5   def initialize(aluno, professor, disciplina, dia, horario)
6     @aluno = aluno
7     #...
8   end
9 end
```


Ruby

1.1 Orientação a Objetos

Uso de dicionários para mapear professores e disciplinas.



```
1 class GerentadorDCM
2   def initialize
3     # Hashes garantem buscas em Tempo Constante O(1)
4     @disciplinas = {
5       "Tinos" => "Estruturas de Dados"
6     }
7     @horarios_permitidos = {
8       "Tinos" => ["Segunda 14h", "Quarta 14h"]
9     }
10    @alunos = {} # Preparado para cadastros dinâmicos
11    @agendamentos = []
12  end
```

Ruby

2. Performance com Hashes

Uso de dicionários para mapear professores e disciplinas.

```
1 @disciplinas = {  
2   "Tinos" => "Estruturas de Dados",  
3   "Baranauskas" => "Paradigmas de Programacao",  
4   "Joaquim" => "Banco de Dados"  
5 }  
6 # Validação em O(1): Acesso direto ao endereço de memória  
7 if @disciplinas [professor] != disciplina  
8   return false  
9 end
```

Ruby

3. Filosofia Fail-Fast

Validação estruturada com Cláusulas de Guarda.

```
1 def agendar(aluno, professor, disciplina, dia, horario)
2   # Cláusula de Guarda: Testa a falha e aborta precocemente
3   if @disciplinas[professor] != disciplina
4     puts "Inválido: disciplina incompatível."
5     return false
6   end
7   # ... valida as outras regras ...
8   # Se sobreviveu a todas as barreiras, o estado é atualizado
9   @agendamentos << Agendamento.new( ... )
10  true
11 end
```


Ruby

4. Iteradores Nativos

Validação estruturada com Cláusulas de Guarda.



```
1 # Substituindo laços de controle de estado manual (for/while)
2 qtd_agendamentos = @agendamentos.count { |a| a.aluno == aluno }
3
4 if qtd_agendamentos >= 2
5     puts "Inválido: limite semanal atingido."
6 end
```



Considerações

O Objetivo é solucionar o problema encontrado em sistemas de agendamentos acadêmicos que são pouco unificados, organizados e acessíveis aos alunos. A solução proposta oferece uma interface fácil de usar que permite ao aluno consultar as agendas de todos os professores e agendar seus horários de monitoria.



OBRIGADO

Enzo Rocha – 15449850
Giovana Leite – 16869044
Murilo Amaral – 17074828

NOSSO REPOSITÓRIO NO GITHUB

